

# 1.0 Introduction to C-Language

C is a **general-purpose, procedural programming language** developed for system and application programming. It is one of the most popular programming languages because it is simple, fast, efficient, and portable. C allows programmers to write structured programs using functions and provides direct access to computer memory.

## Features of C Language:

- Simple and easy to learn.
- Fast execution.
- Portable (runs on different platforms).
- Supports structured programming.
- Rich set of built-in operators and library functions.
- Used for developing operating systems, embedded systems, and application software.

## Applications of C Language:

- Operating systems (Linux, UNIX)
  - Embedded systems
  - System software
  - Database management systems
  - Compilers and interpreters
  - Application software
- 

## 1.1 Describe the History of C-Language and Structure of a C Program

### History of C-Language

C language was developed by **Dennis Ritchie** at **Bell Laboratories** in **1972**. It was developed from an earlier language called **B Language**, created by **Ken Thompson**. C was mainly designed to develop the **UNIX operating system**. Because of its efficiency, portability, and flexibility, C became one of the most widely used programming languages and is considered the foundation of many modern languages like C++, Java, and C#.

## Timeline

- **1967** – BCPL developed by Martin Richards.
- **1970** – B Language developed by Ken Thompson.
- **1972** – C Language developed by Dennis Ritchie.
- **1978** – First C book published by Kernighan and Ritchie.
- **1989** – ANSI standardized C (ANSI C).

## Structure of a C Program

A C program consists of the following sections:

1. **Documentation Section** – Contains comments describing the program.
2. **Link Section** – Includes header files using `#include`.
3. **Definition Section** – Defines symbolic constants using `#define`.
4. **Global Declaration Section** – Declares global variables and function prototypes.
5. **main() Function** – Starting point of program execution.
6. **User-defined Functions** – Additional functions written by the programmer.

## Example Program

```
#include <stdio.h>

int main()
{
    printf("Hello, World!");
    return 0;
}
```

### Explanation:

- `#include <stdio.h>` – Includes the standard input/output library.
- `int main()` – Main function where execution begins.
- `printf()` – Displays output on the screen.
- `return 0;` – Indicates successful execution.

---

## 1.2 Describe the Programming Style of C Language

Programming style refers to the way a C program is written so that it is easy to read, understand, and maintain.

## Guidelines for Good Programming Style

- Use meaningful variable and function names.
- Write proper comments to explain the program.
- Indent the code properly for better readability.
- Use spaces between operators and statements.
- Keep functions small and perform only one task.
- Use uppercase letters for symbolic constants.
- Follow proper naming conventions.
- Avoid unnecessary statements and complex code.
- Use braces `{ }` properly to improve readability.

## Advantages of Good Programming Style

- Easy to understand.
- Easy to debug and maintain.
- Reduces programming errors.
- Improves program readability.
- Makes teamwork and future modifications easier.

### **Conclusion:**

A good programming style helps programmers write clean, organized, and efficient C programs, making them easier to develop, test, and maintain.

## 1.3 Explain the Steps Involved in Editing, Compiling, Executing and Debugging of a C Program

A C program passes through several stages before producing the final output.

### Steps Involved

#### 1. Editing

- Writing the C program using a text editor or IDE (such as Code::Blocks, Turbo C, or VS Code).
- The program is saved with the `.c` extension.

## 2. Compiling

- The compiler checks the program for syntax errors.
- If there are no errors, it converts the source code into object code.

## 3. Linking

- The linker combines the object code with the required library files.
- It creates an executable file.

## 4. Executing (Running)

- The executable file is run by the operating system.
- The program performs the given task and displays the output.

## 5. Debugging

- Debugging is the process of finding and correcting errors in a program.
- Errors may be:
  - **Syntax Errors** – Errors in grammar or syntax.
  - **Runtime Errors** – Errors occurring during execution.
  - **Logical Errors** – Errors in program logic that produce incorrect output.

## Flow of C Program

**Editing → Compiling → Linking → Executing → Debugging (if errors occur)**

### Advantages

- Detects and removes errors.
  - Improves program performance.
  - Produces correct output.
-

## 1.4 Describe Character Set, C-Tokens, Keywords, Identifiers, Constants and Variables

### 1. Character Set

The **character set** is the collection of valid characters used to write a C program.

#### Types of Characters:

- **Alphabets:** A–Z, a–z
  - **Digits:** 0–9
  - **Special Symbols:** +, -, \*, /, %, #, &, @, etc.
  - **White Spaces:** Space, Tab, New Line
- 

### 2. C-Tokens

C-Tokens are the smallest individual units of a C program.

#### Types of C-Tokens:

- Keywords
  - Identifiers
  - Constants
  - Strings
  - Operators
  - Special Symbols
- 

### 3. Keywords

Keywords are **reserved words** in C that have predefined meanings. They cannot be used as variable names.

#### Examples:

- `int`
- `float`
- `char`
- `if`
- `else`

- `for`
  - `while`
  - `return`
- 

## 4. Identifiers

Identifiers are the names given to variables, functions, arrays, etc.

### Rules for Identifiers:

- Must begin with a letter or underscore (`_`).
- Can contain letters, digits, and underscores.
- Cannot be a keyword.
- No spaces or special symbols are allowed.

### Examples:

- `student`
  - `total_marks`
  - `count1`
- 

## 5. Constants

Constants are fixed values that do not change during program execution.

### Types of Constants:

- Integer Constants – `10`, `25`
  - Floating Constants – `3.14`, `5.6`
  - Character Constants – `'A'`, `'5'`
  - String Constants – `"Hello"`
- 

## 6. Variables

Variables are named memory locations used to store data. Their values can change during program execution.

### Example:

```
int age = 18;
float marks = 95.5;
char grade = 'A';
```

### Advantages of Variables

- Store data temporarily.
  - Values can be modified.
  - Make programs flexible and reusable.
- 

## 1.5 Define Data Type

A **data type** specifies the type of data that a variable can store. It also determines the amount of memory allocated and the operations that can be performed on the data.

### Types of Data Types in C

#### 1. Basic (Primary) Data Types

- `int` – Stores integer values.
- `char` – Stores a single character.
- `float` – Stores decimal values.
- `double` – Stores double-precision decimal values.

#### 2. Derived Data Types

- Array
- Pointer
- Function

#### 3. User-Defined Data Types

- `struct`
- `union`
- `enum`
- `typedef`

#### 4. Void Data Type

- `void` represents the absence of a value.
- It is mainly used in functions that do not return any value.

## Example

```
int age = 20;  
float salary = 25000.50;  
char grade = 'A';  
double pi = 3.14159;
```

### Advantages of Data Types

- Allocate appropriate memory.
- Improve program efficiency.
- Prevent invalid data storage.
- Make programs easy to understand and maintain.

## 1.6 Classify Data Types and Explain Them with Examples

A **data type** specifies the type of data that a variable can store. It also determines the memory required and the operations that can be performed on the data.

### Classification of Data Types

#### 1. Basic (Primary) Data Types

These are the fundamental data types provided by C.

- **int** – Stores integer values.
- **char** – Stores a single character.
- **float** – Stores decimal (floating-point) values.
- **double** – Stores double-precision decimal values.

#### Example:

```
int age = 18;  
char grade = 'A';  
float marks = 92.5;  
double pi = 3.141592;
```

---

#### 2. Derived Data Types

These data types are derived from the basic data types.

- Arrays

- Pointers
- Functions

**Example:**

```
int numbers[5];
```

---

### 3. User-Defined Data Types

These are created by the programmer.

- struct
- union
- enum
- typedef

**Example:**

```
struct Student
{
    int roll;
    char name[20];
};
```

---

### 4. Void Data Type

The `void` data type represents the absence of a value. It is mainly used in functions that do not return any value.

**Example:**

```
void display()
{
    printf("Hello");
}
```

### Advantages of Data Types

- Allocate proper memory.
  - Improve program efficiency.
  - Prevent invalid data storage.
  - Make programs easy to understand.
-

# 1.7 Explain Declaration of Constants and Variables

## Variable Declaration

A **variable** is a named memory location used to store data. Before using a variable, it must be declared with a data type.

### Syntax:

```
data_type variable_name;
```

### Examples:

```
int age;  
float salary;  
char grade;
```

---

## Constant Declaration

A **constant** is a fixed value that cannot be changed during program execution.

Constants can be declared using the `const` keyword or the `#define` preprocessor directive.

### Using `const`:

```
const int MAX = 100;
```

### Using `#define`:

```
#define PI 3.14
```

## Difference Between Variable and Constant

| Variable                    | Constant                                                                                        |
|-----------------------------|-------------------------------------------------------------------------------------------------|
| Value can change.           | Value cannot change.                                                                            |
| Declared using a data type. | Declared using <code>const</code> or <code>#define</code> .                                     |
| Occupies memory.            | <code>const</code> occupies memory, while <code>#define</code> is replaced by the preprocessor. |

---

## 1.8 Explain Initializing Values to Variables in Declaration

**Initialization** is the process of assigning a value to a variable at the time of its declaration.

### Syntax:

```
data_type variable_name = value;
```

### Examples:

```
int age = 18;
float marks = 95.5;
char grade = 'A';
```

Multiple variables can also be initialized in a single statement.

### Example:

```
int a = 10, b = 20, c = 30;
```

## Advantages of Initialization

- Prevents the use of garbage (undefined) values.
- Makes the program more reliable.
- Improves readability.
- Reduces programming errors.

### Conclusion:

Initializing variables during declaration is a good programming practice because it ensures that variables contain valid values before they are used in the program.

## 1.9 Explain User-Defined Data Types with a Simple Program

**User-defined data types** are data types created by the programmer to organize and manage data efficiently. C provides the following user-defined data types:

- **Structure** (`struct`)
- **Union** (`union`)
- **Enumeration** (`enum`)
- **Typedef** (`typedef`)

Among these, **structure (struct)** is the most commonly used.

## Advantages of User-Defined Data Types

- Group related data into a single unit.
- Improve program readability.
- Reduce code complexity.
- Make programs easier to maintain.

## Simple Program Using Structure

```
#include <stdio.h>

struct Student
{
    int roll;
    char grade;
};

int main()
{
    struct Student s;

    s.roll = 101;
    s.grade = 'A';

    printf("Roll Number = %d\n", s.roll);
    printf("Grade = %c", s.grade);

    return 0;
}
```

### Output:

```
Roll Number = 101
Grade = A
```

### Explanation:

- `struct Student` defines a new user-defined data type.
  - `s` is a structure variable.
  - Values are assigned using the dot (.) operator.
  - `printf()` displays the stored values.
-

## 1.10 Explain the Usage of Type Qualifiers

**Type qualifiers** are keywords used to modify the properties of data types and variables. They provide additional information to the compiler about how a variable should be used.

### Types of Type Qualifiers

#### 1. const

- Declares a variable whose value cannot be changed after initialization.

**Example:**

```
const int x = 10;
```

---

#### 2. volatile

- Indicates that the value of a variable may change unexpectedly due to external factors such as hardware or interrupts.
- Prevents the compiler from making optimization assumptions about the variable.

**Example:**

```
volatile int sensor;
```

---

#### 3. signed

- Allows a variable to store both positive and negative values.
- It is the default for `int`.

**Example:**

```
signed int num = -50;
```

---

#### 4. unsigned

- Allows a variable to store only non-negative values, increasing the positive range.

**Example:**

```
unsigned int count = 100;
```

## Advantages of Type Qualifiers

- Improve program safety and reliability.
- Protect constant values from modification.
- Support hardware and embedded programming.
- Increase the range of values where required.

### **Conclusion:**

Type qualifiers help control how variables are stored and accessed. They improve the efficiency, safety, and reliability of C programs.

## 2.0 Input and Output Statements, Operators and Expressions in C

Input and Output (I/O) statements are used to communicate between the user and the computer. Input statements accept data from the user, while output statements display the results on the screen. Operators and expressions are used to perform calculations and logical operations.

### Input Functions

- `scanf()` – Reads formatted input.
- `getchar()` – Reads a single character.
- `getch()` – Reads a character without displaying it.
- `getche()` – Reads a character and displays it.

### Output Functions

- `printf()` – Displays formatted output.
- `putchar()` – Displays a single character.
- `putch()` – Displays a single character on the screen.

### Operators

Operators perform operations on variables and constants.

#### Types of Operators:

- Arithmetic Operators (+, -, \*, /, %)
- Relational Operators (>, <, >=, <=, ==, !=)
- Logical Operators (&&, ||, !)
- Assignment Operators (=, +=, -=)
- Increment and Decrement Operators (++ , --)

### Expressions

An expression is a combination of variables, constants, and operators that produces a value.

#### Example:

```
sum = a + b;
```

---

## 2.1 Explain the Importance of Pre-processor Directive

`#include`

The **#include** directive is a preprocessor command used to include header files in a C program before compilation.

### Importance of `#include`

- Includes standard library functions.
- Avoids rewriting predefined functions.
- Makes programming easier and faster.
- Improves code reusability.
- Allows the compiler to recognize library functions.

### Syntax

```
#include <header_file>
```

### Example

```
#include <stdio.h>

int main()
{
    printf("Welcome to C");
    return 0;
}
```

Here, **stdio.h** contains declarations for input/output functions such as `printf()` and `scanf()`.

---

## 2.2.1 Reading a Character Using `getch()`, `getche()` and `getchar()`

These functions are used to read a single character from the keyboard.

### 1. `getch()`

- Reads a character without displaying it on the screen.
- Available in **conio.h**.

### Example:

```
#include <stdio.h>
#include <conio.h>

int main()
{
    char ch;
    ch = getch();
    printf("%c", ch);
    return 0;
}
```

---

## 2. `getche()`

- Reads a character and displays it on the screen.
- Available in `conio.h`.

### Example:

```
#include <stdio.h>
#include <conio.h>

int main()
{
    char ch;
    ch = getche();
    return 0;
}
```

---

## 3. `getchar()`

- Reads a single character and displays it on the screen.
- Available in `stdio.h`.

### Example:

```
#include <stdio.h>

int main()
{
    char ch;
    ch = getchar();
    printf("%c", ch);
    return 0;
}
```

---

## 2.2.2 Writing a Character Using `putch()` and `putchar()`

These functions display a single character on the screen.

### 1. `putch()`

- Writes one character to the screen.
- Declared in `conio.h`.

#### Example:

```
#include <conio.h>

int main()
{
    putch('A');
    return 0;
}
```

---

### 2. `putchar()`

- Writes one character to the screen.
- Declared in `stdio.h`.

#### Example:

```
#include <stdio.h>

int main()
{
    putchar('B');
    return 0;
}
```

---

## 2.2.3 Formatted Input Using `scanf()` with Sample Program

The `scanf()` function is used to read formatted input from the user.

### Syntax

```
scanf("format_specifier", &variable);
```

## Example Program

```
#include <stdio.h>

int main()
{
    int age;
    printf("Enter Age: ");
    scanf("%d", &age);
    printf("Age = %d", age);
    return 0;
}
```

### Explanation:

- `scanf()` reads the value entered by the user.
  - `%d` is the format specifier for an integer.
  - `&age` gives the address of the variable.
- 

## 2.2.4 Formatted Output Using `printf()` with Sample Program

The **`printf()`** function is used to display formatted output on the screen.

### Syntax

```
printf("format_string", variable);
```

## Example Program

```
#include <stdio.h>

int main()
{
    int marks = 95;
    printf("Marks = %d", marks);
    return 0;
}
```

### Explanation:

- `printf()` displays output on the screen.
- `%d` displays an integer value.
- Different format specifiers are used for different data types.

### Common Format Specifiers

| Format Specifier | Data Type                             |
|------------------|---------------------------------------|
| %d               | Integer ( <code>int</code> )          |
| %f               | Floating-point ( <code>float</code> ) |
| %c               | Character ( <code>char</code> )       |
| %s               | String                                |
| %lf              | Double ( <code>double</code> )        |

### Advantages of `printf()` and `scanf()`

- Easy to use for input and output.
- Support different data types.
- Make programs interactive.
- Widely used in C programming.

## 2.3 Explain Character Functions

Character functions are predefined functions used to test and convert characters. These functions are declared in the `ctype.h` header file.

### Common Character Functions

#### 1. `isalpha()`

- Checks whether a character is an alphabet.

#### Example:

```
isalpha('A');
```

---

#### 2. `isdigit()`

- Checks whether a character is a digit.

#### Example:

```
isdigit('5');
```

---

### 3. `isalnum()`

- Checks whether a character is an alphabet or a digit.

#### **Example:**

```
isalnum('B');
```

---

### 4. `islower()`

- Checks whether a character is a lowercase letter.

#### **Example:**

```
islower('a');
```

---

### 5. `isupper()`

- Checks whether a character is an uppercase letter.

#### **Example:**

```
isupper('A');
```

---

### 6. `tolower()`

- Converts an uppercase letter to lowercase.

#### **Example:**

```
tolower('A');
```

---

### 7. `toupper()`

- Converts a lowercase letter to uppercase.

#### **Example:**

```
toupper('a');
```

## Advantages of Character Functions

- Simplify character handling.
  - Improve program readability.
  - Reduce coding time.
  - Useful in text processing.
- 

## 2.4 Define an Operator and an Expression

### Operator

An **operator** is a symbol that performs a specific operation on one or more operands.

#### Examples:

- + (Addition)
  - - (Subtraction)
  - \* (Multiplication)
  - / (Division)
  - % (Modulus)
- 

### Expression

An **expression** is a combination of variables, constants, and operators that produces a value.

#### Example:

```
sum = a + b;
```

Here,  $a + b$  is an arithmetic expression.

### Types of Expressions

- Arithmetic Expression
- Relational Expression
- Logical Expression
- Assignment Expression

---

## 2.5.1 Various Arithmetic Operators and Evaluation of Arithmetic Expressions

Arithmetic operators are used to perform mathematical calculations.

| Operator | Operation           |
|----------|---------------------|
| +        | Addition            |
| -        | Subtraction         |
| *        | Multiplication      |
| /        | Division            |
| %        | Modulus (Remainder) |

### Example Program

```
#include <stdio.h>

int main()
{
    int a = 10, b = 3;

    printf("Addition = %d\n", a + b);
    printf("Subtraction = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division = %d\n", a / b);
    printf("Modulus = %d", a % b);

    return 0;
}
```

### Evaluation of Arithmetic Expressions

The compiler evaluates arithmetic expressions according to **operator precedence**.

#### Order of Evaluation:

1. Parentheses ()
2. Multiplication \*, Division /, Modulus %
3. Addition +, Subtraction -

**Example:**

```
x = 5 + 2 * 3;
```

**Result:** 11 (Multiplication is performed before addition.)

---

## 2.5.2 Various Relational Operators and Evaluation of Relational Expressions

Relational operators are used to compare two values. The result is either **True (1)** or **False (0)**.

| Operator | Meaning                  |
|----------|--------------------------|
| >        | Greater than             |
| <        | Less than                |
| >=       | Greater than or equal to |
| <=       | Less than or equal to    |
| ==       | Equal to                 |
| !=       | Not equal to             |

### Example

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20;

    printf("%d\n", a < b);
    printf("%d", a == b);

    return 0;
}
```

### Evaluation

- 10 < 20 → **1 (True)**
- 10 == 20 → **0 (False)**

## Applications

- Used in decision-making statements.
  - Used in loops and conditional statements.
- 

## 2.5.3 Various Logical Operators and Evaluation of Logical Expressions

Logical operators are used to combine two or more conditions.

### Operator    Meaning

&&            Logical AND

,

!             Logical NOT

### Example Program

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20;

    printf("%d\n", (a < b && b > 5));
    printf("%d\n", (a > b || b > 5));
    printf("%d", !(a < b));

    return 0;
}
```

### Evaluation

- (a < b && b > 5) → **1 (True)** because both conditions are true.
- (a > b || b > 5) → **1 (True)** because one condition is true.
- !(a < b) → **0 (False)** because the NOT operator reverses the result.

### Advantages of Logical Operators

- Combine multiple conditions.

- Simplify decision-making.
- Used in `if`, `while`, `for`, and other control statements.
- Improve program efficiency.

## 2.6 Explain the Difference Between Unary and Binary Operators

Operators are symbols used to perform operations on operands. Based on the number of operands they operate on, they are classified as **Unary Operators** and **Binary Operators**.

### Unary Operators

A **Unary Operator** works on **one operand** only.

#### Examples:

- `++` (Increment)
- `--` (Decrement)
- `-` (Unary Minus)
- `!` (Logical NOT)

#### Example:

```
int a = 10;  
a++;
```

Here, `++` operates on only one operand (`a`).

---

### Binary Operators

A **Binary Operator** works on **two operands**.

#### Examples:

- `+` (Addition)
- `-` (Subtraction)
- `*` (Multiplication)
- `/` (Division)

- % (Modulus)
- = (Assignment)

### Example:

```
int c = a + b;
```

Here, + operates on two operands (a and b).

## Difference Between Unary and Binary Operators

| Unary Operator           | Binary Operator                         |
|--------------------------|-----------------------------------------|
| Operates on one operand. | Operates on two operands.               |
| Examples: ++, --, !      | Examples: +, -, *, /                    |
| Simpler operations.      | Performs operations between two values. |

---

## 2.7 Describe Various Assignment Operators, Increment and Decrement Operators

### Assignment Operators

Assignment operators are used to assign values to variables.

| Operator | Meaning             | Example |
|----------|---------------------|---------|
| =        | Assign              | a = 10; |
| +=       | Add and assign      | a += 5; |
| -=       | Subtract and assign | a -= 2; |
| *=       | Multiply and assign | a *= 3; |
| /=       | Divide and assign   | a /= 2; |
| %=       | Modulus and assign  | a %= 2; |

**Example:**

```
int a = 10;  
a += 5;  
printf("%d", a);
```

---

## Increment Operator (++)

The increment operator increases the value of a variable by **1**.

**Types:**

- **Pre-increment (++a)** – Increments first, then uses the value.
- **Post-increment (a++)** – Uses the value first, then increments.

**Example:**

```
int a = 5;  
a++;
```

---

## Decrement Operator (--)

The decrement operator decreases the value of a variable by **1**.

**Types:**

- **Pre-decrement (--a)**
- **Post-decrement (a--)**

**Example:**

```
int a = 5;  
a--;
```

## Advantages

- Reduce the number of statements.
  - Make programs shorter and easier to understand.
  - Improve coding efficiency.
-

## 2.8 Illustrate Nested Assignment

**Nested assignment** means assigning a value to multiple variables in a single statement. The assignment is performed from **right to left**.

### Syntax

```
a = b = c = value;
```

### Example Program

```
#include <stdio.h>

int main()
{
    int a, b, c;

    a = b = c = 100;

    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("c = %d", c);

    return 0;
}
```

### Output:

```
a = 100
b = 100
c = 100
```

### Explanation

- First, the value 100 is assigned to c.
- Then the same value is assigned to b.
- Finally, it is assigned to a.

### Advantages of Nested Assignment

- Reduces the number of statements.
- Makes the code shorter and simpler.
- Improves readability when assigning the same value to multiple variables.

## 2.9 Explain Conditional Operator with an Example

The **Conditional Operator** (`?:`) is also called the **Ternary Operator** because it uses **three operands**. It is used to make simple decisions and is a shorthand form of the `if-else` statement.

### Syntax

```
condition ? expression1 : expression2;
```

- If the **condition is true**, `expression1` is executed.
- If the **condition is false**, `expression2` is executed.

### Example Program

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20, max;

    max = (a > b) ? a : b;

    printf("Maximum = %d", max);

    return 0;
}
```

### Output:

```
Maximum = 20
```

### Advantages

- Reduces the number of lines of code.
- Makes programs simple and easy to understand.
- Replaces simple `if-else` statements.

---

## 2.10.1 Bit-wise Operators and Explain Each with an Example

**Bit-wise operators** perform operations on the individual bits of integer values.

## Operator    Meaning

|    |             |
|----|-------------|
| &  | Bitwise AND |
|    | Bitwise OR  |
| ^  | Bitwise XOR |
| ~  | Bitwise NOT |
| << | Left Shift  |
| >> | Right Shift |

## Example Program

```
#include <stdio.h>

int main()
{
    int a = 5, b = 3;

    printf("AND = %d\n", a & b);
    printf("OR = %d\n", a | b);
    printf("XOR = %d\n", a ^ b);
    printf("Left Shift = %d\n", a << 1);
    printf("Right Shift = %d", a >> 1);

    return 0;
}
```

## Uses of Bit-wise Operators

- Embedded systems programming.
- Device drivers.
- Fast arithmetic operations.
- Data manipulation.

---

## 2.10.2 Special Operators with Examples

Special operators perform specific operations in C.

## 1. sizeof Operator

Returns the size of a data type or variable in bytes.

### Example:

```
#include <stdio.h>

int main()
{
    printf("%d", sizeof(int));
    return 0;
}
```

---

## 2. Comma Operator (,)

Allows multiple expressions in a single statement.

### Example:

```
int a = 5, b = 10;
```

---

## 3. Pointer Operator (\*)

Used to declare and access pointers.

### Example:

```
int *p;
```

---

## 4. Address Operator (&)

Returns the memory address of a variable.

### Example:

```
int a = 10;
printf("%p", &a);
```

---

## 2.10.3 Precedence and Associativity of Operators

### Operator Precedence

**Precedence** determines the order in which operators are evaluated in an expression.

#### Highest to Lowest Precedence

1. `()` (Parentheses)
2. Unary Operators `++, --, !`
3. `*`, `/`, `%`
4. `+`, `-`
5. Relational Operators `<`, `>`, `<=`, `>=`
6. Equality Operators `==`, `!=`
7. Logical AND `&&`
8. Logical OR `||`
9. Assignment `=`

### Associativity

Associativity determines the order of evaluation when operators have the same precedence.

- **Left to Right:** `+`, `-`, `*`, `/`, `%`
- **Right to Left:** Assignment `=`, Unary Operators

#### Example:

```
x = 5 + 2 * 3;
```

**Result:** 11

---

## 2.11 Describe Evaluation of Compound Expression

A **compound expression** is an expression that contains **more than one operator**. It is evaluated according to the **precedence** and **associativity** rules.

### Example

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 5, c = 2;
    int result;

    result = a + b * c;

    printf("Result = %d", result);

    return 0;
}
```

### Output:

Result = 20

### Explanation

- First,  $b * c = 5 \times 2 = 10$
- Then,  $a + 10 = 20$
- Final result = **20**

### Advantages of Compound Expressions

- Reduce the number of statements.
- Make programs shorter and efficient.
- Perform multiple operations in a single expression.
- Improve program readability when written correctly.

## 2.12 Illustrate Type Conversion Techniques

**Type conversion** is the process of converting a value from one data type to another. It helps perform operations between different data types.

### Types of Type Conversion

#### 1. Implicit Type Conversion (Automatic Conversion)

In this type, the compiler automatically converts one data type into another.

#### Example:

```
#include <stdio.h>
```

```
int main()
{
    int a = 10;
    float b = 5.5;

    float sum = a + b;

    printf("Sum = %.1f", sum);

    return 0;
}
```

### Output:

Sum = 15.5

### Explanation:

- The integer value 10 is automatically converted into 10.0.
- The result is stored as a floating-point value.

---

## 2. Explicit Type Conversion (Type Casting)

In this type, the programmer manually converts one data type into another using the **type cast operator**.

### Syntax:

```
(data_type) expression;
```

### Example:

```
#include <stdio.h>

int main()
{
    int a = 10, b = 3;
    float result;

    result = (float)a / b;

    printf("Result = %.2f", result);

    return 0;
}
```

### Output:

Result = 3.33

## Advantages of Type Conversion

- Prevents data loss.
  - Improves calculation accuracy.
  - Allows operations between different data types.
  - Makes programs more flexible.
- 

## 2.13 Write Sample Programs Using All the Operators

### Program 1: Arithmetic Operators

```
#include <stdio.h>

int main()
{
    int a = 10, b = 5;

    printf("Addition = %d\n", a + b);
    printf("Subtraction = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division = %d\n", a / b);
    printf("Modulus = %d", a % b);

    return 0;
}
```

---

### Program 2: Relational Operators

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20;

    printf("%d\n", a < b);
    printf("%d\n", a > b);
    printf("%d", a == b);

    return 0;
}
```

---

### Program 3: Logical Operators

```
#include <stdio.h>

int main()
{
```

```
int a = 10, b = 20;

printf("%d\n", (a < b && b > 5));
printf("%d\n", (a > b || b > 5));
printf("%d", !(a < b));

return 0;
}
```

---

## Program 4: Assignment Operators

```
#include <stdio.h>

int main()
{
    int a = 10;

    a += 5;
    printf("%d\n", a);

    a -= 2;
    printf("%d", a);

    return 0;
}
```

---

## Program 5: Increment and Decrement Operators

```
#include <stdio.h>

int main()
{
    int a = 5;

    printf("%d\n", ++a);
    printf("%d\n", a++);
    printf("%d\n", --a);
    printf("%d", a--);

    return 0;
}
```

---

## Program 6: Conditional Operator

```
#include <stdio.h>

int main()
{
    int a = 15, b = 20;

    int max = (a > b) ? a : b;
```

```
    printf("Maximum = %d", max);

    return 0;
}
```

---

## Program 7: Bitwise Operators

```
#include <stdio.h>

int main()
{
    int a = 5, b = 3;

    printf("AND = %d\n", a & b);
    printf("OR = %d\n", a | b);
    printf("XOR = %d", a ^ b);

    return 0;
}
```

---

## Program 8: sizeof Operator

```
#include <stdio.h>

int main()
{
    printf("Size of int = %d", sizeof(int));

    return 0;
}
```

## Conclusion

These sample programs demonstrate the use of the major operators in C, including **arithmetic, relational, logical, assignment, increment/decrement, conditional, bitwise, and special operators**. They help in understanding how different operators are used to perform various operations in C programming.

## 3.0 Decision Making, Iterative and Other Control Statements

Control statements determine the flow of execution in a C program. They help in making decisions, repeating statements, and transferring control from one part of the program to another.

### Types of Control Statements

#### 1. Decision Making Statements

- o if
- o if-else
- o nested if-else
- o if-else-if ladder
- o switch

#### 2. Iterative (Looping) Statements

- o while
- o do-while
- o for

#### 3. Jump Statements

- o break
- o continue
- o goto
- o return

### Advantages

- Controls program execution.
- Reduces code repetition.
- Makes programs efficient and flexible.
- Supports decision making and looping.

---

## 3.1 Explain Decision Making Statements and Their Need in Programming

**Decision making statements** are used to execute different blocks of code based on a condition. They help the program choose one action from multiple alternatives.

## Need for Decision Making

- Executes statements based on conditions.
- Helps solve real-life problems.
- Reduces unnecessary execution of statements.
- Improves program efficiency.
- Makes programs intelligent and flexible.

### Examples of Decision Making Statements

- `if`
  - `if-else`
  - `nested if-else`
  - `if-else-if ladder`
  - `switch`
- 

## 3.2.1 Simple `if` and `if-else` Statement with Syntax and Sample Program

### Simple `if` Statement

The **`if` statement** executes a block of code only when the given condition is true.

#### Syntax

```
if(condition)
{
    statements;
}
```

#### Sample Program

```
#include <stdio.h>

int main()
{
    int age = 20;

    if(age >= 18)
    {
        printf("Eligible to Vote");
    }

    return 0;
}
```

---

## `if-else` Statement

The **if-else statement** executes one block if the condition is true and another block if the condition is false.

### Syntax

```
if(condition)
{
    statements1;
}
else
{
    statements2;
}
```

### Sample Program

```
#include <stdio.h>

int main()
{
    int num = 7;

    if(num % 2 == 0)
        printf("Even Number");
    else
        printf("Odd Number");

    return 0;
}
```

---

## 3.2.2 Nested `if-else` Statements with Syntax and Sample Program

A **Nested if-else** means an if or if-else statement inside another if or else block.

### Syntax

```
if(condition1)
{
    if(condition2)
    {
        statements;
    }
    else
    {
```

```

        statements;
    }
}
else
{
    statements;
}

```

## Sample Program

```

#include <stdio.h>

int main()
{
    int a = 20, b = 15;

    if(a > 10)
    {
        if(a > b)
            printf("A is Greater");
        else
            printf("B is Greater");
    }

    return 0;
}

```

---

### 3.2.3 `if-else-if` Ladder with Syntax and Sample Program

The **`if-else-if ladder`** is used when multiple conditions need to be checked one after another.

#### Syntax

```

if(condition1)
{
    statements;
}
else if(condition2)
{
    statements;
}
else if(condition3)
{
    statements;
}
else
{
    statements;
}

```

## Sample Program

```

#include <stdio.h>

```

```

int main()
{
    int marks = 75;

    if(marks >= 90)
        printf("Grade A");
    else if(marks >= 75)
        printf("Grade B");
    else if(marks >= 50)
        printf("Grade C");
    else
        printf("Fail");

    return 0;
}

```

---

### 3.2.4 `switch` Statement with Syntax and Sample Program

The **switch statement** is used to select one block of code from multiple choices based on the value of an expression.

#### Syntax

```

switch(expression)
{
    case value1:
        statements;
        break;

    case value2:
        statements;
        break;

    default:
        statements;
}

```

#### Sample Program

```

#include <stdio.h>

int main()
{
    int day = 2;

    switch(day)
    {
        case 1:
            printf("Monday");
            break;
    }
}

```

```

        case 2:
            printf("Tuesday");
            break;

        case 3:
            printf("Wednesday");
            break;

        default:
            printf("Invalid Day");
    }

    return 0;
}

```

### Advantages of `switch` Statement

- Easy to read and understand.
- Suitable for multiple choices.
- Improves program readability.
- Faster than a long `if-else-if` ladder in many cases.

## 3.3 State the Importance of `break` Statement with `switch` and Illustrate

The **`break`** statement is used inside a **`switch`** statement to terminate the execution of the current case and transfer control to the statement following the `switch`.

### Importance of `break` Statement

- Terminates the current case.
- Prevents execution of the next cases (fall-through).
- Improves program efficiency.
- Makes the program easy to understand.

### Example Program

```

#include <stdio.h>

int main()
{
    int choice = 2;

    switch(choice)
    {

```

```

        case 1:
            printf("One");
            break;

        case 2:
            printf("Two");
            break;

        case 3:
            printf("Three");
            break;

        default:
            printf("Invalid Choice");
    }

    return 0;
}

```

### Output:

Two

### Explanation:

- Since `choice = 2`, **Case 2** is executed.
- The `break` statement stops execution, so **Case 3** is not executed.

## 3.4.1 Compare Conditional Operator with `if-else` Statement

### Conditional Operator (?:)

Uses the `?:` operator.

Suitable for simple conditions.

Short and compact.

Returns a value.

Used in expressions.

### `if-else` Statement

Uses `if` and `else` keywords.

Suitable for simple and complex conditions.

Longer but easier to understand.

Executes a block of statements.

Used for decision making in programs.

### Example (Conditional Operator):

```
max = (a > b) ? a : b;
```

### Example (if-else):

```
if(a > b)
    max = a;
else
    max = b;
```

---

## 3.4.2 Compare `if-else` with `switch` Statement

| <code>if-else</code> Statement            | <code>switch</code> Statement                          |
|-------------------------------------------|--------------------------------------------------------|
| Used for single or multiple conditions.   | Used for multiple choices based on one expression.     |
| Can use relational and logical operators. | Uses only constant values in <code>case</code> labels. |
| Suitable for complex conditions.          | Suitable for menu-driven programs.                     |
| Slower when many conditions exist.        | Faster for multiple fixed choices.                     |
| Easier for range checking.                | Cannot check ranges directly.                          |

---

## 3.5 Define Looping or Iteration

**Looping** or **Iteration** is the process of executing a group of statements repeatedly until a specified condition becomes false.

Loops reduce the need to write the same statements multiple times and make programs shorter and more efficient.

### Types of Loops in C

1. **while Loop**
2. **do-while Loop**
3. **for Loop**

## Advantages of Looping

- Reduces code repetition.
- Saves programming time.
- Makes programs shorter and easier to maintain.
- Improves program efficiency.

### Example:

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 1; i <= 5; i++)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

### Output:

```
1
2
3
4
5
```

### Explanation:

The `for` loop executes five times and prints the numbers from **1 to 5**.

## 3.6 List and Explain Iterative Statements with Syntax and Examples

**Iterative statements** (loops) are used to execute a block of statements repeatedly until a specified condition becomes false.

### Types of Iterative Statements

1. `while` Loop

2. `do-while` Loop
  3. `for` Loop
- 

## 1. `while` Loop

The **`while`** loop checks the condition before executing the loop body. It is also called an **entry-controlled loop**.

### Syntax

```
while(condition)
{
    statements;
}
```

### Example Program

```
#include <stdio.h>

int main()
{
    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

---

## 2. `do-while` Loop

The **`do-while`** loop executes the loop body first and then checks the condition. It is also called an **exit-controlled loop**.

### Syntax

```
do
{
    statements;
}
while(condition);
```

### Example Program

```
#include <stdio.h>
```

```
int main()
{
    int i = 1;

    do
    {
        printf("%d\n", i);
        i++;
    }
    while(i <= 5);

    return 0;
}
```

---

### 3. `for` Loop

The **`for`** loop is used when the number of iterations is known in advance.

#### Syntax

```
for(initialization; condition; increment/decrement)
{
    statements;
}
```

#### Example Program

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 1; i <= 5; i++)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

#### Advantages of Iterative Statements

- Reduce code repetition.
  - Improve program readability.
  - Save programming time.
  - Make programs efficient.
-

## 3.7 Compare Different Loop Statements

| while Loop                          | do-while Loop                                      | for Loop                                                       |
|-------------------------------------|----------------------------------------------------|----------------------------------------------------------------|
| Entry-controlled loop.              | Exit-controlled loop.                              | Entry-controlled loop.                                         |
| Condition checked before execution. | Condition checked after execution.                 | Initialization, condition, and increment are in one statement. |
| May execute zero times.             | Executes at least one time.                        | Best when the number of iterations is known.                   |
| Suitable for unknown iterations.    | Suitable when the loop must execute at least once. | Suitable for fixed iterations.                                 |

---

## 3.8 What is Nested Loop? Illustrate

A **Nested Loop** is a loop placed inside another loop. The inner loop executes completely for each iteration of the outer loop.

### Syntax

```
for(initialization; condition; increment)
{
    for(initialization; condition; increment)
    {
        statements;
    }
}
```

### Example Program

```
#include <stdio.h>

int main()
{
    int i, j;

    for(i = 1; i <= 3; i++)
    {
        for(j = 1; j <= 3; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}
```

```
    return 0;
}
```

### Output:

```
* * *
* * *
* * *
```

### Explanation

- The **outer loop** runs **3 times**.
- For each outer loop iteration, the **inner loop** also runs **3 times**.
- As a result, a **3 × 3** pattern of stars is printed.

### Applications of Nested Loops

- Printing patterns.
- Matrix operations.
- Table generation.
- Processing two-dimensional arrays.

## 3.9 Explain the Usage of `goto`, `break` and `continue` Statements with Loop Statements

### 1. `goto` Statement

The **goto** statement transfers program control directly to a labeled statement within the same function.

#### Syntax

```
goto label;
...
label:
    statements;
```

#### Example Program

```
#include <stdio.h>

int main()
{
```

```
    int i = 1;

label:
    printf("%d\n", i);
    i++;

    if(i <= 5)
        goto label;

    return 0;
}
```

### Uses of `goto`

- Transfers control to a specified label.
  - Used to exit deeply nested loops (rarely used).
  - Should be avoided in large programs because it reduces readability.
- 

## 2. `break` Statement

The **`break`** statement immediately terminates the loop and transfers control to the statement following the loop.

### Example Program

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 1; i <= 10; i++)
    {
        if(i == 6)
            break;

        printf("%d\n", i);
    }

    return 0;
}
```

### Output:

```
1
2
3
4
5
```

---

### 3. `continue` Statement

The **`continue`** statement skips the remaining statements in the current iteration and starts the next iteration of the loop.

#### Example Program

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 1; i <= 5; i++)
    {
        if(i == 3)
            continue;

        printf("%d\n", i);
    }

    return 0;
}
```

#### Output:

```
1
2
4
5
```

#### Advantages

- `goto` transfers control to any labeled statement.
- `break` exits the loop immediately.
- `continue` skips the current iteration and continues with the next iteration.

---

## 3.10 Differentiate `break` and `continue` Statements

### **`break` Statement**

Terminates the loop immediately.

Control moves outside the loop.

### **`continue` Statement**

Skips the current iteration and continues with the next iteration.

Control remains inside the loop.

### **break Statement**

Used in loops and `switch` statements.

Stops further execution of the loop.

### **continue Statement**

Used only in loops.

Does not stop the loop; only skips one iteration.

#### Example of `break`

```
if(i == 5)
    break;
```

#### Example of `continue`

```
if(i == 5)
    continue;
```

---

## 3.11 Define Structured Programming

**Structured Programming** is a programming approach in which a program is divided into small, logical, and manageable modules or functions. It uses **sequence, selection, and iteration** to solve problems in a systematic way.

### Features of Structured Programming

- Divides a program into functions or modules.
- Uses control statements (`if`, `switch`, `for`, `while`).
- Avoids unnecessary use of `goto`.
- Improves readability and maintainability.
- Makes debugging and testing easier.

### Advantages of Structured Programming

- Easy to understand.
- Easy to debug and maintain.
- Reduces program complexity.
- Encourages code reuse.
- Improves program reliability.

### Examples of Structured Programming Languages

- C
- Pascal
- Ada

**Conclusion:**

Structured programming helps develop programs in a clear, organized, and modular manner, making them easier to read, test, and maintain.

## 4.0 Arrays, Strings, Structures and Unions

Arrays, strings, structures, and unions are important data structures in C. They are used to store and organize data efficiently.

- **Array** – Stores multiple values of the same data type.
- **String** – An array of characters ending with the null character ('\\0').
- **Structure** – Groups different data types under one name.
- **Union** – Similar to a structure, but all members share the same memory location.

### Advantages

- Efficient data storage.
  - Easy access to data.
  - Reduces code complexity.
  - Improves program organization.
- 

## 4.1 Define Array

An **Array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Each element is identified by an **index**, starting from **0**.

### Syntax

```
data_type array_name[size];
```

### Example

```
int marks[5];
```

Here, `marks` is an array that can store **5 integer values**.

### Advantages of Arrays

- Store multiple values using one variable.
  - Easy to access elements using an index.
  - Reduces the number of variables.
  - Simplifies data processing.
-

## 4.2.1 Declaration and Initialization of One-Dimensional (1D) Array with Syntax and Sample Program

### Declaration

A **One-Dimensional (1D) Array** stores elements in a single row.

#### Syntax

```
data_type array_name[size];
```

#### Example:

```
int marks[5];
```

---

### Initialization

Values can be assigned to the array at the time of declaration.

#### Syntax

```
data_type array_name[size] = {values};
```

#### Example:

```
int marks[5] = {80, 85, 90, 95, 100};
```

---

### Sample Program

```
#include <stdio.h>

int main()
{
    int marks[5] = {80, 85, 90, 95, 100};
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("%d\n", marks[i]);
    }

    return 0;
}
```

---

## 4.2.2 Accessing the Elements in 1D Array with Sample Program

Array elements are accessed using their **index number**. The index starts from **0**.

### Syntax

```
array_name[index];
```

### Sample Program

```
#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("%d\n", a[i]);
    }

    return 0;
}
```

### Output:

```
10
20
30
40
50
```

### Explanation

- a[0] stores **10**
  - a[1] stores **20**
  - a[2] stores **30**
  - a[3] stores **40**
  - a[4] stores **50**
- 

## 4.2.3 Reordering an Array in Ascending Order

**Ascending order** means arranging the elements from the **smallest value to the largest value**.

## Algorithm

1. Read the array elements.
2. Compare each element with the remaining elements.
3. If the first element is greater than the second, swap them.
4. Repeat until the entire array is sorted.
5. Display the sorted array.

## Sample Program

```
#include <stdio.h>

int main()
{
    int a[5] = {25, 10, 40, 15, 30};
    int i, j, temp;

    for(i = 0; i < 5; i++)
    {
        for(j = i + 1; j < 5; j++)
        {
            if(a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    printf("Array in Ascending Order:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

### Output:

```
Array in Ascending Order:
10 15 25 30 40
```

## Advantages of Sorting an Array

- Makes searching easier.
- Organizes data systematically.
- Improves program efficiency.
- Simplifies data processing.

## 4.0 Arrays, Strings, Structures and Unions

Arrays, strings, structures, and unions are important data structures in C. They are used to store and organize data efficiently.

- **Array** – Stores multiple values of the same data type.
- **String** – An array of characters ending with the null character ('\0').
- **Structure** – Groups different data types under one name.
- **Union** – Similar to a structure, but all members share the same memory location.

### Advantages

- Efficient data storage.
  - Easy access to data.
  - Reduces code complexity.
  - Improves program organization.
- 

## 4.1 Define Array

An **Array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Each element is identified by an **index**, starting from **0**.

### Syntax

```
data_type array_name[size];
```

### Example

```
int marks[5];
```

Here, `marks` is an array that can store **5 integer values**.

### Advantages of Arrays

- Store multiple values using one variable.
  - Easy to access elements using an index.
  - Reduces the number of variables.
  - Simplifies data processing.
-

## 4.2.1 Declaration and Initialization of One-Dimensional (1D) Array with Syntax and Sample Program

### Declaration

A **One-Dimensional (1D) Array** stores elements in a single row.

#### Syntax

```
data_type array_name[size];
```

#### Example:

```
int marks[5];
```

---

### Initialization

Values can be assigned to the array at the time of declaration.

#### Syntax

```
data_type array_name[size] = {values};
```

#### Example:

```
int marks[5] = {80, 85, 90, 95, 100};
```

---

### Sample Program

```
#include <stdio.h>

int main()
{
    int marks[5] = {80, 85, 90, 95, 100};
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("%d\n", marks[i]);
    }

    return 0;
}
```

---

## 4.2.2 Accessing the Elements in 1D Array with Sample Program

Array elements are accessed using their **index number**. The index starts from **0**.

### Syntax

```
array_name[index];
```

### Sample Program

```
#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("%d\n", a[i]);
    }

    return 0;
}
```

### Output:

```
10
20
30
40
50
```

### Explanation

- a[0] stores **10**
  - a[1] stores **20**
  - a[2] stores **30**
  - a[3] stores **40**
  - a[4] stores **50**
- 

## 4.2.3 Reordering an Array in Ascending Order

**Ascending order** means arranging the elements from the **smallest value to the largest value**.

## Algorithm

1. Read the array elements.
2. Compare each element with the remaining elements.
3. If the first element is greater than the second, swap them.
4. Repeat until the entire array is sorted.
5. Display the sorted array.

## Sample Program

```
#include <stdio.h>

int main()
{
    int a[5] = {25, 10, 40, 15, 30};
    int i, j, temp;

    for(i = 0; i < 5; i++)
    {
        for(j = i + 1; j < 5; j++)
        {
            if(a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    printf("Array in Ascending Order:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

### Output:

```
Array in Ascending Order:
10 15 25 30 40
```

## Advantages of Sorting an Array

- Makes searching easier.
- Organizes data systematically.
- Improves program efficiency.
- Simplifies data processing.

## 4.3 Explain Declaration, Initialization and Usage of Two-Dimensional (2D) Arrays

A **Two-Dimensional (2D) Array** is an array of arrays. It is used to store data in the form of **rows and columns** (matrix).

### Declaration

#### Syntax

```
data_type array_name[rows][columns];
```

#### Example:

```
int a[3][3];
```

---

### Initialization

Values can be assigned at the time of declaration.

#### Example:

```
int a[2][2] = {{1,2},{3,4}};
```

---

### Usage of 2D Arrays

- Used to represent matrices.
- Used in mathematical calculations.
- Used in tables and grids.
- Used in image processing.

### Sample Program

```
#include <stdio.h>

int main()
{
    int a[2][2] = {{10,20},{30,40}};
    int i,j;

    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
```

```
        printf("%d ",a[i][j]);
    }
    printf("\n");
}

return 0;
}
```

---

## 4.4 Illustrate the Concept of Arrays with Sample Programs on Matrix Addition, Subtraction and Matrix Multiplication

### 1. Matrix Addition

```
#include <stdio.h>

int main()
{
    int a[2][2]={{1,2},{3,4}};
    int b[2][2]={{5,6},{7,8}};
    int c[2][2];
    int i,j;

    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            c[i][j]=a[i][j]+b[i][j];
            printf("%d ",c[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

---

### 2. Matrix Subtraction

```
#include <stdio.h>

int main()
{
    int a[2][2]={{8,6},{4,2}};
    int b[2][2]={{1,2},{3,4}};
    int c[2][2];
    int i,j;

    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            c[i][j]=a[i][j]-b[i][j];
        }
    }
}
```

```
        printf("%d ",c[i][j]);
    }
    printf("\n");
}
return 0;
}
```

---

### 3. Matrix Multiplication

```
#include <stdio.h>

int main()
{
    int a[2][2]={1,2},{3,4};
    int b[2][2]={5,6},{7,8};
    int c[2][2];
    int i,j,k;

    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            c[i][j]=0;
            for(k=0;k<2;k++)
            {
                c[i][j]=c[i][j]+a[i][k]*b[k][j];
            }
            printf("%d ",c[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

---

## 4.5 Define String

A **String** is a collection of characters stored in a character array and terminated by the **null character** (`'\0'`).

### Syntax

```
char string_name[size];
```

### Example:

```
char name[20] = "Srikar";
```

## Advantages

- Stores multiple characters.
  - Easy to process text data.
  - Supports various string functions.
- 

## 4.6.1 Declare and Initialize String Variables

A string variable is declared using the **char** data type.

### Declaration

```
char name[20];
```

### Initialization

```
char name[20] = "Hello";
```

or

```
char name[] = {'H','e','l','l','o','\0'};
```

### Sample Program

```
#include <stdio.h>

int main()
{
    char name[20] = "C Language";

    printf("%s", name);

    return 0;
}
```

---

## 4.6.2 `gets()` and `puts()`

### `gets()` Function

- Reads a complete string from the keyboard until the Enter key is pressed.

### Syntax

```
gets(string_name);
```

---

## `puts()` Function

- Displays a string on the screen.

### Syntax

```
puts(string_name);
```

### Sample Program

```
#include <stdio.h>

int main()
{
    char name[30];

    gets(name);
    puts(name);

    return 0;
}
```

---

## 4.6.3 Reading and Displaying Strings from Terminal with Sample Program

Strings can be read using **gets()** and displayed using **puts()**.

### Sample Program

```
#include <stdio.h>

int main()
{
    char str[30];

    printf("Enter a String: ");
    gets(str);

    printf("Entered String: ");
    puts(str);

    return 0;
}
```

---

## 4.6.4 Explain Various String Handling Functions with Sample Programs

String handling functions are declared in the `string.h` header file.

### 1. `strlen()`

Returns the length of a string.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "Hello";

    printf("%d", strlen(str));

    return 0;
}
```

---

### 2. `strcpy()`

Copies one string into another.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[20], s2[] = "C Language";

    strcpy(s1, s2);

    printf("%s", s1);

    return 0;
}
```

---

### 3. `strcat()`

Concatenates (joins) two strings.

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char s1[30] = "Hello ";
    char s2[] = "World";

    strcat(s1, s2);

    printf("%s", s1);

    return 0;
}
```

---

#### 4. strcmp()

Compares two strings.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[] = "ABC";
    char s2[] = "ABC";

    printf("%d", strcmp(s1, s2));

    return 0;
}
```

### Common String Functions

| Function | Purpose |
|----------|---------|
|----------|---------|

|          |                              |
|----------|------------------------------|
| strlen() | Finds the length of a string |
|----------|------------------------------|

|          |                              |
|----------|------------------------------|
| strcpy() | Copies one string to another |
|----------|------------------------------|

|          |                   |
|----------|-------------------|
| strcat() | Joins two strings |
|----------|-------------------|

|          |                      |
|----------|----------------------|
| strcmp() | Compares two strings |
|----------|----------------------|

### Advantages of String Handling Functions

- Simplify string operations.
- Reduce programming effort.
- Improve code readability.
- Save development time.

## 4.7 Explain Character Arithmetic

**Character arithmetic** is the process of performing arithmetic operations on character values using their **ASCII (American Standard Code for Information Interchange)** values.

In C, each character is stored as an integer (ASCII code), so arithmetic operations can be performed on characters.

### Example

```
#include <stdio.h>

int main()
{
    char ch = 'A';

    printf("%c\n", ch);
    printf("%c\n", ch + 1);
    printf("%d", ch);

    return 0;
}
```

#### Output:

```
A
B
65
```

#### Explanation:

- ASCII value of 'A' is **65**.
- 'A' + 1 becomes **66**, which represents 'B'.

### Applications

- Character conversion.
- Encryption techniques.
- Text processing.

---

## 4.8 Define a Structure

A **Structure** is a user-defined data type that groups **different types of data** under a single name.

It allows storing related data items together.

## Syntax

```
struct structure_name
{
    data_type member1;
    data_type member2;
};
```

## Example

```
struct Student
{
    int roll;
    char grade;
    float marks;
};
```

## Advantages

- Groups related data.
  - Improves readability.
  - Simplifies data management.
  - Supports complex data structures.
- 

## 4.9.1 Initializing Structure, Declaring Structure and Declaring Structure Variables

### Structure Declaration

```
struct Student
{
    int roll;
    char grade;
};
```

---

### Declaring Structure Variable

```
struct Student s1;
```

---

### Initializing Structure

```
struct Student s1 = {101, 'A'};
```

## Sample Program

```
#include <stdio.h>

struct Student
{
    int roll;
    char grade;
};

int main()
{
    struct Student s1 = {101, 'A'};

    printf("%d %c", s1.roll, s1.grade);

    return 0;
}
```

---

## 4.9.2 Accessing the Structure Members

Structure members are accessed using the **dot (.) operator**.

### Syntax

```
structure_variable.member_name;
```

### Example Program

```
#include <stdio.h>

struct Student
{
    int roll;
    char grade;
};

int main()
{
    struct Student s;

    s.roll = 101;
    s.grade = 'A';

    printf("%d\n", s.roll);
    printf("%c", s.grade);

    return 0;
}
```

---

## 4.9.3 Structure Assignment

One structure variable can be assigned to another structure variable of the **same type**.

### Example Program

```
#include <stdio.h>

struct Student
{
    int roll;
};

int main()
{
    struct Student s1 = {101}, s2;

    s2 = s1;

    printf("%d", s2.roll);

    return 0;
}
```

---

## 4.9.4 How to Find the Size of a Structure

The **sizeof** operator is used to determine the size of a structure.

### Example Program

```
#include <stdio.h>

struct Student
{
    int roll;
    char grade;
};

int main()
{
    printf("%d", sizeof(struct Student));

    return 0;
}
```

---

## 4.9.5 Nested Structure Concept

A **Nested Structure** is a structure that contains another structure as one of its members.

### Example Program

```
#include <stdio.h>

struct Address
{
    char city[20];
};

struct Student
{
    int roll;
    struct Address add;
};

int main()
{
    struct Student s;

    s.roll = 101;

    printf("%d", s.roll);

    return 0;
}
```

### Advantages

- Organizes complex data.
  - Improves readability.
  - Reduces code complexity.
- 

## 4.9.6 Structures Containing Arrays

A structure can contain one or more arrays as its members.

### Example Program

```
#include <stdio.h>

struct Student
{
    char name[20];
```

```

        int marks[3];
    };

int main()
{
    struct Student s = {"Ravi", {80, 85, 90}};

    printf("%s\n", s.name);
    printf("%d", s.marks[0]);

    return 0;
}

```

## Applications

- Student records.
  - Employee details.
  - Database applications.
- 

## 4.9.7 Array of Structures

An **Array of Structures** is an array in which each element is a structure variable.

### Syntax

```
struct Student s[5];
```

### Sample Program

```

#include <stdio.h>

struct Student
{
    int roll;
};

int main()
{
    struct Student s[2] = {{101}, {102}};

    printf("%d\n", s[0].roll);
    printf("%d", s[1].roll);

    return 0;
}

```

### Output:

101

## Advantages of Array of Structures

- Stores multiple records efficiently.
- Easy to access using array indexing.
- Reduces the number of variables.
- Widely used in student, employee, and library management systems.

## 4.10 Define Union, Declare, Initialize and Use of Union

A **Union** is a user-defined data type similar to a structure. In a union, **all members share the same memory location**, so only one member can hold a value at a time.

### Syntax

```
union union_name
{
    data_type member1;
    data_type member2;
};
```

### Declaration

```
union Student
{
    int roll;
    float marks;
};
```

### Declaring a Union Variable

```
union Student s;
```

### Initializing a Union

```
union Student s = {101};
```

### Sample Program

```
#include <stdio.h>

union Student
{
    int roll;
    float marks;
};

int main()
```

```

{
    union Student s;

    s.roll = 101;
    printf("Roll = %d\n", s.roll);

    s.marks = 95.5;
    printf("Marks = %.1f", s.marks);

    return 0;
}

```

## Advantages of Union

- Saves memory.
- Efficient memory utilization.
- Useful when only one data member is used at a time.
- Widely used in embedded systems and device programming.

## 4.11 Distinguish Between Structures and Unions

| Structure                                           | Union                                            |
|-----------------------------------------------------|--------------------------------------------------|
| Each member has its own memory location.            | All members share the same memory location.      |
| All members can store values simultaneously.        | Only one member can store a value at a time.     |
| Size is the sum of all member sizes (plus padding). | Size is equal to the size of the largest member. |
| Uses more memory.                                   | Uses less memory.                                |
| Suitable when all members are required together.    | Suitable when only one member is used at a time. |

## 4.12 Write Sample Programs for All the Concepts of Structures and Unions

### 1. Structure Program

```
#include <stdio.h>

struct Student
{
    int roll;
    char grade;
};

int main()
{
    struct Student s = {101, 'A'};

    printf("Roll = %d\n", s.roll);
    printf("Grade = %c", s.grade);

    return 0;
}
```

---

### 2. Nested Structure Program

```
#include <stdio.h>

struct Address
{
    char city[20];
};

struct Student
{
    int roll;
    struct Address add;
};

int main()
{
    struct Student s = {101, "Hyderabad"};

    printf("%d\n", s.roll);
    printf("%s", s.add.city);

    return 0;
}
```

---

### 3. Structure Containing an Array

```
#include <stdio.h>
```

```
struct Student
{
    char name[20];
    int marks[3];
};

int main()
{
    struct Student s = {"Ravi", {80, 85, 90}};

    printf("%s\n", s.name);
    printf("%d", s.marks[0]);

    return 0;
}
```

---

## 4. Array of Structures

```
#include <stdio.h>

struct Student
{
    int roll;
};

int main()
{
    struct Student s[2] = {{101}, {102}};

    printf("%d\n", s[0].roll);
    printf("%d", s[1].roll);

    return 0;
}
```

---

## 5. Union Program

```
#include <stdio.h>

union Student
{
    int roll;
    float marks;
};

int main()
{
    union Student s;

    s.roll = 101;
    printf("Roll = %d\n", s.roll);

    s.marks = 92.5;
```

```
    printf("Marks = %.1f", s.marks);  
  
    return 0;  
}
```

---

## 6. Finding Size of a Structure

```
#include <stdio.h>  
  
struct Student  
{  
    int roll;  
    char grade;  
};  
  
int main()  
{  
    printf("Size = %d", sizeof(struct Student));  
  
    return 0;  
}
```

## Conclusion

Structures and unions are user-defined data types used to organize data efficiently. **Structures** allocate separate memory for each member and allow all members to store values simultaneously, whereas **unions** share a single memory location among all members, making them memory-efficient. They are widely used in software development, database applications, and embedded systems.

## 5.0 User Defined Functions, Pointers, File Management and Preprocessor Directives

C language provides **user-defined functions**, **pointers**, **file management**, and **preprocessor directives** to develop efficient, modular, and reusable programs.

- **User-defined Functions** – Functions created by the programmer to perform specific tasks.
- **Pointers** – Variables that store the memory address of another variable.
- **File Management** – Used to store and retrieve data permanently using files.
- **Preprocessor Directives** – Commands processed before compilation, such as `#include` and `#define`.

### Advantages

- Improves code reusability.
  - Reduces program complexity.
  - Saves memory and execution time.
  - Makes programs modular and easy to maintain.
- 

## 5.1 Explain

### 1. Need of User Defined Functions

A **User Defined Function (UDF)** is a function created by the programmer to perform a specific task.

#### Need of User Defined Functions

- Breaks a large program into smaller modules.
  - Avoids repetition of code.
  - Improves program readability.
  - Simplifies debugging and testing.
  - Makes the program easy to maintain and modify.
-

## 2. Advantages of Functions

Functions provide many benefits in C programming.

### Advantages

- Reduce code duplication.
  - Improve program readability.
  - Make debugging easier.
  - Support code reusability.
  - Simplify program maintenance.
  - Facilitate teamwork by dividing the program into modules.
- 

## 3. Elements of a Function

Every function consists of the following elements:

### 1. Function Declaration (Prototype)

Specifies the function name, return type, and parameters.

#### Example:

```
int add(int, int);
```

---

### 2. Function Definition

Contains the actual code of the function.

#### Example:

```
int add(int a, int b)
{
    return a + b;
}
```

---

### 3. Function Call

Invokes the function to execute its statements.

**Example:**

```
sum = add(10, 20);
```

---

## 4. Return Values and Their Types

A function may or may not return a value to the calling function.

### Types of Return Values

#### 1. Function Returning a Value

Uses the `return` statement.

**Example:**

```
int square(int n)
{
    return n * n;
}
```

---

#### 2. Function Returning No Value

Uses the `void` return type.

**Example:**

```
void display()
{
    printf("Hello");
}
```

### Advantages of Return Values

- Pass results back to the calling function.
  - Improve modular programming.
  - Increase code reusability.
-

## 5.2 Define Function Call and Function Prototype

### Function Call

A **Function Call** is the statement that transfers control from the calling function to the called function.

#### Syntax

```
function_name(arguments);
```

#### Example

```
sum = add(10, 20);
```

---

### Function Prototype

A **Function Prototype** is a declaration that tells the compiler about the function's **name, return type, and parameters** before it is used.

#### Syntax

```
return_type function_name(parameter_list);
```

#### Example

```
int add(int, int);
```

### Advantages of Function Prototype

- Informs the compiler about the function.
- Detects errors before compilation.
- Improves program readability.
- Supports modular programming.

### Difference Between Function Prototype and Function Call

| Function Prototype                                   | Function Call                     |
|------------------------------------------------------|-----------------------------------|
| Declares the function.                               | Invokes the function.             |
| Contains function name, return type, and parameters. | Passes arguments to the function. |

| Function Prototype                    | Function Call                                   |
|---------------------------------------|-------------------------------------------------|
| Executed before the function is used. | Executes the function during program execution. |
| Helps the compiler check errors.      | Transfers control to the called function.       |

## 5.3 Explain Functions

Functions are blocks of code that perform a specific task. They help divide a large program into smaller and manageable modules.

---

### 5.3.1 Function Declaration in Programs

A **function declaration** (prototype) tells the compiler about the function name, return type, and parameters before it is used.

#### Syntax

```
return_type function_name(parameter_list);
```

#### Example

```
int add(int, int);
```

#### Sample Program

```
#include <stdio.h>

int add(int, int);

int main()
{
    int sum;
    sum = add(10,20);
    printf("Sum = %d", sum);
    return 0;
}

int add(int a, int b)
{
    return a+b;
}
```

### Advantages

- Detects errors before compilation.
  - Improves readability.
  - Supports modular programming.
- 

## 5.3.2 Functions with No Arguments and No Return Values

These functions neither accept arguments nor return a value.

### Syntax

```
void function_name()
{
    statements;
}
```

### Sample Program

```
#include <stdio.h>

void display();

int main()
{
    display();
    return 0;
}

void display()
{
    printf("Welcome to C Programming");
}
```

### Advantages

- Simple to use.
  - Suitable for displaying messages.
- 

## 5.3.3 Functions with Arguments and No Return Values

These functions receive arguments but do not return any value.

## Syntax

```
void function_name(data_type variable)
{
    statements;
}
```

## Sample Program

```
#include <stdio.h>

void square(int);

int main()
{
    square(5);
    return 0;
}

void square(int n)
{
    printf("Square = %d", n*n);
}
```

## Advantages

- Accepts user data.
  - No need to return a value.
- 

## 5.3.4 Functions with Arguments and Return Values

These functions receive arguments and return a value.

## Syntax

```
return_type function_name(parameters)
{
    statements;
    return value;
}
```

## Sample Program

```
#include <stdio.h>

int add(int,int);

int main()
{
```

```

    int sum;

    sum = add(10,20);

    printf("Sum = %d", sum);

    return 0;
}

int add(int a,int b)
{
    return a+b;
}

```

### Advantages

- Produces a result.
- Improves code reusability.
- Widely used in programming.

## 5.3.5 Functions with No Arguments and Return Values

These functions do not receive any arguments but return a value.

### Syntax

```

return_type function_name()
{
    statements;
    return value;
}

```

### Sample Program

```

#include <stdio.h>

int getNumber();

int main()
{
    int num;

    num = getNumber();

    printf("Number = %d", num);

    return 0;
}

```

```
int getNumber()
{
    return 100;
}
```

### Advantages

- Returns a value without accepting arguments.
  - Simple and reusable.
  - Useful when input is not required.
- 

## 5.3.6 Functions that Return Multiple Values with Sample Program

A function can return only one value directly. To return multiple values, **pointers** or **structures** are used.

### Sample Program

```
#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int s, p;

    calculate(10, 20, &s, &p);

    printf("Sum = %d\n", s);
    printf("Product = %d", p);

    return 0;
}
```

### Advantages

- Returns multiple values.
- Reduces the number of function calls.
- Improves efficiency.

---

## 5.3.7 Recursion with Sample Program

**Recursion** is a process in which a function calls itself until a stopping condition is reached.

### Syntax

```
return_type function_name()
{
    if(condition)
        return value;
    else
        function_name();
}
```

### Sample Program (Factorial)

```
#include <stdio.h>

int fact(int n)
{
    if(n == 1)
        return 1;
    else
        return n * fact(n - 1);
}

int main()
{
    printf("%d", fact(5));
    return 0;
}
```

### Output

120

### Advantages

- Makes programs shorter.
- Useful for factorial, Fibonacci, and tree problems.

---

## 5.3.8 Passing Arrays to Functions with Sample Program

Arrays can be passed to functions by passing the array name.

## Syntax

```
function_name(array_name);
```

## Sample Program

```
#include <stdio.h>

void display(int a[])
{
    int i;

    for(i = 0; i < 5; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[5] = {10,20,30,40,50};

    display(a);

    return 0;
}
```

## Advantages

- Avoids duplication of code.
- Easy processing of array elements.

---

## 5.3.9 Structure as Function Arguments and Structure as Function Values

Structures can be passed to functions as arguments and can also be returned from functions.

## Sample Program

```
#include <stdio.h>

struct Student
{
    int roll;
};

void display(struct Student s)
{
    printf("%d", s.roll);
}
```

```
int main()
{
    struct Student s = {101};

    display(s);

    return 0;
}
```

### Advantages

- Passes complete records.
  - Improves modular programming.
- 

## 5.3.10 Structures Containing Pointers

A structure can contain pointer variables as its members.

### Example

```
#include <stdio.h>

struct Student
{
    int *ptr;
};

int main()
{
    int a = 100;

    struct Student s;

    s.ptr = &a;

    printf("%d", *s.ptr);

    return 0;
}
```

### Applications

- Dynamic memory allocation.
- Linked lists.
- Trees and graphs.

---

## 5.3.11 Self-Referential Structures with Example

A **Self-Referential Structure** is a structure that contains a pointer to the same type of structure.

### Syntax

```
struct Node
{
    int data;
    struct Node *next;
};
```

### Sample Program

```
#include <stdio.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node n1;

    n1.data = 10;
    n1.next = NULL;

    printf("%d", n1.data);

    return 0;
}
```

### Applications

- Linked Lists.
- Stacks.
- Queues.
- Trees.

### Advantages

- Supports dynamic data structures.
- Efficient memory utilization.
- Easy insertion and deletion of elements.

---

## 5.3.12 Storage Classes – `auto`, `register`, `static`, `extern`

A **Storage Class** defines the **scope, visibility, lifetime, and storage location** of a variable in a C program.

### Types of Storage Classes

#### 1. `auto` Storage Class

- It is the default storage class for local variables.
- Variables are created when the function starts and destroyed when the function ends.

##### Example:

```
#include <stdio.h>

int main()
{
    auto int a = 10;
    printf("%d", a);

    return 0;
}
```

---

#### 2. `register` Storage Class

- Requests the compiler to store the variable in a CPU register for faster access.
- Mainly used for frequently accessed variables.

##### Example:

```
#include <stdio.h>

int main()
{
    register int i;

    for(i = 1; i <= 5; i++)
        printf("%d ", i);

    return 0;
}
```

---

### 3. `static` Storage Class

- Retains the value of a variable between function calls.
- Initialized only once.

#### Example:

```
#include <stdio.h>

void display()
{
    static int count = 0;
    count++;
    printf("%d\n", count);
}

int main()
{
    display();
    display();
    display();

    return 0;
}
```

#### Output:

```
1
2
3
```

---

### 4. `extern` Storage Class

- Declares a global variable that is defined in another file or outside the current function.
- Used to share variables between files.

#### Example:

```
#include <stdio.h>

extern int a;

int a = 100;

int main()
{
    printf("%d", a);

    return 0;
}
```

}

## Advantages of Storage Classes

- Control the lifetime of variables.
  - Improve memory management.
  - Increase program efficiency.
  - Support modular programming.
- 

## 5.3.13 Scope, Visibility and Lifetime of Variables in Functions

### 1. Scope

The **scope** of a variable is the region of the program where it can be accessed.

**Types:**

- Local Scope
  - Global Scope
- 

### 2. Visibility

**Visibility** refers to the parts of the program where a variable is accessible.

- Local variables are visible only inside their function.
  - Global variables are visible throughout the program.
- 

### 3. Lifetime

The **lifetime** of a variable is the duration for which it exists in memory.

- Local variables exist only while the function is executing.
- Global and `static` variables exist throughout the program execution.

## Advantages

- Proper memory utilization.
  - Better program organization.
  - Easy debugging and maintenance.
- 

## 5.4 Differentiate Local and External Variables

| Local Variable                                  | External (Global) Variable                                              |
|-------------------------------------------------|-------------------------------------------------------------------------|
| Declared inside a function.                     | Declared outside all functions.                                         |
| Accessible only within that function.           | Accessible throughout the program.                                      |
| Created when the function starts.               | Exists throughout program execution.                                    |
| Cannot be accessed by other functions directly. | Can be accessed by all functions using <code>extern</code> if required. |
| Stored in local memory.                         | Stored in global memory.                                                |

---

## 5.5 Define Global Variable

A **Global Variable** is a variable declared **outside all functions**. It can be accessed by every function in the program.

### Syntax

```
data_type variable_name;
```

### Example Program

```
#include <stdio.h>

int a = 100;

void display()
{
    printf("%d\n", a);
}
```

```
int main()
{
    printf("%d\n", a);
    display();

    return 0;
}
```

### Output:

```
100
100
```

## Advantages of Global Variables

- Accessible throughout the program.
  - Reduces the need to pass variables as function arguments.
  - Suitable for sharing common data among functions.
  - Simplifies large programs.
- 

## 5.7 Explain Pointers

A **Pointer** is a variable that stores the **memory address** of another variable. Pointers help in efficient memory management and are widely used in functions, arrays, structures, and dynamic memory allocation.

---

### 5.7.1 Declaration and Initialization of Pointers

A pointer is declared using the \* operator.

#### Syntax

```
data_type *pointer_name;
```

#### Example

```
int *p;
```

#### Initialization

```
int a = 10;
int *p = &a;
```

## Sample Program

```
#include <stdio.h>

int main()
{
    int a = 10;
    int *p = &a;

    printf("%d", *p);

    return 0;
}
```

---

## 5.7.2 Accessing the Address of a Variable Using & Operator

The **address operator (&)** returns the memory address of a variable.

### Syntax

```
&variable_name;
```

## Sample Program

```
#include <stdio.h>

int main()
{
    int a = 100;

    printf("%p", &a);

    return 0;
}
```

### Explanation:

- `&a` gives the memory address of variable `a`.
- 

## 5.7.3 Accessing the Value of a Variable Through Pointer

The **dereference operator (\*)** is used to access the value stored at the address contained in a pointer.

## Sample Program

```
#include <stdio.h>

int main()
{
    int a = 50;
    int *p = &a;

    printf("%d", *p);

    return 0;
}
```

### Output

50

---

## 5.7.4 Pointer Arithmetic

Pointer arithmetic is used to move pointers to different memory locations.

### Operations

- Increment (++)
- Decrement (--)
- Addition (+)
- Subtraction (-)

## Sample Program

```
#include <stdio.h>

int main()
{
    int a[3]={10,20,30};
    int *p=a;

    printf("%d\n", *p);
    p++;
    printf("%d", *p);

    return 0;
}
```

### Output

10

## 5.7.5 Precedence of Address and Dereferencing Operators

- **& (Address Operator)** has higher precedence than assignment.
- **\* (Dereference Operator)** accesses the value stored at an address.

### Example

```
#include <stdio.h>

int main()
{
    int a=10;
    int *p=&a;

    printf("%d", *p);

    return 0;
}
```

### Explanation

- `&a` gives the address of `a`.
  - `*p` gives the value stored at that address.
- 

## 5.7.6 Relationship Between Arrays and Pointers

The name of an array represents the address of its first element.

For an array:

```
int a[5];
```

- `a` is equivalent to `&a[0]`.

### Sample Program

```
#include <stdio.h>

int main()
{
    int a[3]={10,20,30};
```

```
    printf("%d\n", *a);
    printf("%d", *(a+1));

    return 0;
}
```

### Output

```
10
20
```

---

## 5.7.7 Accessing Array Elements Using Pointers

Array elements can be accessed using pointer arithmetic.

### Sample Program

```
#include <stdio.h>

int main()
{
    int a[5]={1,2,3,4,5};
    int *p=a;
    int i;

    for(i=0;i<5;i++)
    {
        printf("%d ", *(p+i));
    }

    return 0;
}
```

---

## 5.7.8 Pointers as Function Arguments

Pointers can be passed as function arguments to modify the original variable.

### Sample Program

```
#include <stdio.h>

void display(int *x)
{
    *x=*x+10;
}
```

```
int main()
{
    int a=20;

    display(&a);

    printf("%d", a);

    return 0;
}
```

## Output

30

## Advantages

- Saves memory.
- Allows modification of original data.
- Useful for arrays and structures.

---

## 5.7.9 Pointer Arrays with Examples

A **Pointer Array** is an array whose elements are pointers.

### Syntax

```
data_type *array_name[size];
```

### Example

```
int *p[3];
```

### Sample Program

```
#include <stdio.h>

int main()
{
    int a=10,b=20,c=30;

    int *p[3];

    p[0]=&a;
    p[1]=&b;
    p[2]=&c;

    printf("%d\n", *p[0]);
}
```

```

    printf("%d\n", *p[1]);
    printf("%d", *p[2]);

    return 0;
}

```

## Output

```

10
20
30

```

## Advantages of Pointers

- Efficient memory management.
- Supports dynamic memory allocation.
- Enables call by reference.
- Used in arrays, structures, linked lists, trees, and file handling.
- Improves program execution efficiency.

## 5.8 Differentiate Between Address and Dereferencing Operators

| Address Operator (&)                      | Dereferencing Operator (*)                             |
|-------------------------------------------|--------------------------------------------------------|
| Returns the memory address of a variable. | Returns the value stored at the memory address.        |
| Called the <b>address-of operator</b> .   | Called the <b>indirection (dereference) operator</b> . |
| Used to obtain the address of a variable. | Used to access the value pointed to by a pointer.      |
| Example: <code>p = &amp;a;</code>         | Example: <code>*p</code>                               |

### Example Program

```

#include <stdio.h>

int main()
{
    int a = 10;
    int *p = &a;
}

```

```
printf("Address = %p\n", &a);  
printf("Value = %d", *p);  
  
return 0;  
}
```

---

## 5.9 Explain

### 5.9.1 Dynamic Memory Management Functions with Examples

**Dynamic Memory Management** is the process of allocating and freeing memory during program execution. These functions are declared in the `stdlib.h` header file.

#### Dynamic Memory Functions

##### 1. `malloc()`

Allocates a block of memory.

##### Syntax

```
pointer = (data_type *)malloc(size);
```

##### 2. `calloc()`

Allocates memory for multiple elements and initializes them to zero.

##### Syntax

```
pointer = (data_type *)calloc(n, size);
```

##### 3. `realloc()`

Changes the size of previously allocated memory.

##### Syntax

```
pointer = realloc(pointer, new_size);
```

## 4. `free()`

Releases dynamically allocated memory.

### Syntax

```
free(pointer);
```

### Sample Program

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;

    p = (int *)malloc(sizeof(int));

    *p = 100;

    printf("%d", *p);

    free(p);

    return 0;
}
```

### Advantages

- Efficient memory usage.
  - Memory is allocated only when needed.
  - Reduces memory wastage.
- 

## 5.9.2 Structures Containing Pointers

A structure can have pointer members that store the addresses of variables.

### Example Program

```
#include <stdio.h>

struct Student
{
    int *ptr;
};
```

```
int main()
{
    int a = 50;

    struct Student s;

    s.ptr = &a;

    printf("%d", *s.ptr);

    return 0;
}
```

## Applications

- Dynamic memory allocation.
  - Linked lists.
  - Trees and graphs.
- 

## 5.9.3 Pointer to Structure

A **pointer to a structure** stores the address of a structure variable. The **arrow operator (->)** is used to access structure members through a pointer.

### Syntax

```
struct structure_name *pointer_name;
```

### Sample Program

```
#include <stdio.h>

struct Student
{
    int roll;
};

int main()
{
    struct Student s = {101};
    struct Student *p = &s;

    printf("%d", p->roll);

    return 0;
}
```

### Output

## Advantages

- Easy access to structure members.
  - Saves memory.
  - Useful for dynamic data structures.
- 

## 5.9.4 Self-Referential Structures with Examples

A **Self-Referential Structure** is a structure that contains a pointer to the same type of structure.

### Syntax

```
struct Node
{
    int data;
    struct Node *next;
};
```

### Sample Program

```
#include <stdio.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node n1;

    n1.data = 10;
    n1.next = NULL;

    printf("%d", n1.data);

    return 0;
}
```

### Applications

- Linked Lists.
- Stacks.
- Queues.

- Trees.

## Advantages

- Supports dynamic memory allocation.
- Efficient memory utilization.
- Easy insertion and deletion of nodes.
- Widely used in data structures.

## 5.10 Explain File Management

**File Management** in C is used to store data permanently in files. The file handling functions are available in the `stdio.h` header file.

---

### 5.10.1 Files and How to Declare File Pointer to a File

A **File** is a collection of related data stored permanently on a storage device.

A **File Pointer** is a pointer of type `FILE` used to access a file.

## Syntax

```
FILE *fp;
```

## Example

```
#include <stdio.h>

int main()
{
    FILE *fp;

    return 0;
}
```

## Advantages

- Permanent storage of data.
- Easy retrieval of information.
- Reduces data loss.

---

## 5.10.2 Illustrate the Concept of File Opening Using Various Modes

The `fopen()` function is used to open a file.

### Syntax

```
fp = fopen("filename", "mode");
```

### Common File Modes

| Mode | Description                                                                     |
|------|---------------------------------------------------------------------------------|
| "r"  | Opens an existing file for reading.                                             |
| "w"  | Creates a new file or overwrites an existing file for writing.                  |
| "a"  | Opens a file for appending data.                                                |
| "r+" | Opens a file for both reading and writing.                                      |
| "w+" | Opens a file for reading and writing (creates a new file if it does not exist). |
| "a+" | Opens a file for reading and appending.                                         |

### Sample Program

```
#include <stdio.h>

int main()
{
    FILE *fp;

    fp = fopen("test.txt", "w");

    if(fp == NULL)
        printf("File cannot be opened");
    else
        printf("File opened successfully");

    fclose(fp);

    return 0;
}
```

---

## 5.10.3 Illustrate the Concept of Closing a File

The **fclose()** function closes an opened file.

### Syntax

```
fclose(fp);
```

### Sample Program

```
#include <stdio.h>

int main()
{
    FILE *fp;

    fp = fopen("test.txt", "w");

    fclose(fp);

    printf("File Closed");

    return 0;
}
```

### Advantages

- Saves data properly.
  - Frees memory.
  - Prevents data corruption.
- 

## 5.10.4 Illustrate the Concept of Input / Output Operations on a File

Input and output operations are performed using file handling functions.

### Common Functions

- **fprintf()** – Writes formatted data to a file.
- **fscanf()** – Reads formatted data from a file.
- **fputc()** – Writes a character.
- **fgetc()** – Reads a character.
- **fputs()** – Writes a string.

- `fgets()` – Reads a string.

## Sample Program

```
#include <stdio.h>

int main()
{
    FILE *fp;

    fp = fopen("test.txt", "w");

    fprintf(fp, "Welcome to C");

    fclose(fp);

    return 0;
}
```

---

## 5.10.5 Illustrate the Concept of Random Accessing Files

**Random Access** allows reading or writing data at any position in a file without reading the entire file.

### Functions Used

- `fseek()`
- `ftell()`
- `rewind()`

## Sample Program

```
#include <stdio.h>

int main()
{
    FILE *fp;

    fp = fopen("test.txt", "w+");

    fprintf(fp, "ABCDEFGH");

    fseek(fp, 2, SEEK_SET);

    printf("%ld", ftell(fp));

    fclose(fp);

    return 0;
}
```

```
}
```

## Output

2

## Advantages

- Fast access to data.
  - Saves processing time.
  - Efficient for large files.
- 

## 5.10.6 Explain Different File Handling Functions

The following are commonly used file handling functions in C.

| Function               | Purpose                                              |
|------------------------|------------------------------------------------------|
| <code>fopen()</code>   | Opens a file.                                        |
| <code>fclose()</code>  | Closes a file.                                       |
| <code>fprintf()</code> | Writes formatted data to a file.                     |
| <code>fscanf()</code>  | Reads formatted data from a file.                    |
| <code>fputc()</code>   | Writes a single character.                           |
| <code>fgetc()</code>   | Reads a single character.                            |
| <code>fputs()</code>   | Writes a string to a file.                           |
| <code>fgets()</code>   | Reads a string from a file.                          |
| <code>fseek()</code>   | Moves the file pointer to a specified position.      |
| <code>ftell()</code>   | Returns the current position of the file pointer.    |
| <code>rewind()</code>  | Moves the file pointer to the beginning of the file. |
| <code>remove()</code>  | Deletes a file.                                      |

| Function | Purpose |
|----------|---------|
|----------|---------|

|                        |                 |
|------------------------|-----------------|
| <code>rename ()</code> | Renames a file. |
|------------------------|-----------------|

## Advantages of File Handling

- Provides permanent storage.
- Efficient handling of large amounts of data.
- Supports reading, writing, and updating files.
- Enables random access to data.
- Reduces memory usage by storing data externally.

## 5.11 Explain Preprocessor Directives

The **Preprocessor** is a program that processes the source code before the compiler starts compilation. **Preprocessor directives** begin with the # symbol and do not end with a semicolon.

---

### 5.11.1 Preprocessor Directives

A **Preprocessor Directive** is an instruction given to the preprocessor to perform tasks such as including header files, defining constants, and conditional compilation.

#### Common Preprocessor Directives

| Directive | Purpose |
|-----------|---------|
|-----------|---------|

|                       |                         |
|-----------------------|-------------------------|
| <code>#include</code> | Includes a header file. |
|-----------------------|-------------------------|

|                      |                              |
|----------------------|------------------------------|
| <code>#define</code> | Defines constants or macros. |
|----------------------|------------------------------|

|                     |                                       |
|---------------------|---------------------------------------|
| <code>#undef</code> | Undefines a previously defined macro. |
|---------------------|---------------------------------------|

|                  |                                         |
|------------------|-----------------------------------------|
| <code>#if</code> | Executes code if the condition is true. |
|------------------|-----------------------------------------|

|                     |                                    |
|---------------------|------------------------------------|
| <code>#ifdef</code> | Checks whether a macro is defined. |
|---------------------|------------------------------------|

|                      |                                        |
|----------------------|----------------------------------------|
| <code>#ifndef</code> | Checks whether a macro is not defined. |
|----------------------|----------------------------------------|

| Directive           | Purpose                         |
|---------------------|---------------------------------|
| <code>#else</code>  | Specifies an alternative block. |
| <code>#elif</code>  | Specifies another condition.    |
| <code>#endif</code> | Ends a conditional directive.   |

### Example

```
#include <stdio.h>
#define PI 3.14

int main()
{
    printf("%.2f", PI);

    return 0;
}
```

---

## 5.11.2 Need of Preprocessor Directives

Preprocessor directives are used to simplify programming and improve code organization.

### Need of Preprocessor Directives

- Include standard header files.
- Define symbolic constants.
- Create reusable macros.
- Perform conditional compilation.
- Improve program readability.
- Reduce code repetition.
- Make programs easier to maintain.

### Advantages

- Faster compilation.
  - Easy code maintenance.
  - Better program organization.
  - Increases code reusability.
-

## 5.12 Write

### 5.12.1 Simple Programs Using Preprocessor Directives

#### Program 1: Using `#include`

```
#include <stdio.h>

int main()
{
    printf("Welcome to C Programming");

    return 0;
}
```

---

#### Program 2: Using `#define`

```
#include <stdio.h>

#define PI 3.14

int main()
{
    float r = 5;

    printf("Area = %.2f", PI * r * r);

    return 0;
}
```

---

#### Program 3: Using `#ifdef`

```
#include <stdio.h>

#define VALUE 100

int main()
{
#ifdef VALUE
    printf("VALUE is Defined");
#endif

    return 0;
}
```

---

## 5.12.2 Simple Program Using Command Line Arguments

### (argc and argv)

**Command Line Arguments** allow values to be passed to a program while executing it.

- **argc (Argument Count):** Stores the number of command-line arguments.
- **argv (Argument Vector):** Stores the arguments as an array of strings.

### Syntax

```
int main(int argc, char *argv[])
{
    // statements
}
```

### Sample Program

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i;

    printf("Number of Arguments = %d\n", argc);

    for(i = 0; i < argc; i++)
    {
        printf("%s\n", argv[i]);
    }

    return 0;
}
```

### Sample Output

If the program is executed as:

```
program Hello C
```

#### Output:

```
Number of Arguments = 3
program
Hello
C
```

### Advantages of Command Line Arguments

- Pass input values during program execution.

- Avoids hardcoding input in the program.
  - Useful for file names and configuration parameters.
  - Makes programs more flexible and reusable.
-